

M08 Learner Book

Files, APIs & Data Ingestion | Release Candidate 1 | 17 controlled lessons

M08-L01 - Ingestion architecture: source -> landing -> staging

Learning objective

Define source, immutable landing, staging, quarantine and state boundaries.

Why this matters in Data Engineering

Ingestion is not "read file -> dataframe"; trustworthy systems preserve what arrived and separate interpretation from evidence.

Understand

- Landing keeps raw evidence for replay/forensics.
- Staging applies schema/types/normalization at an explicit grain.
- Quarantine preserves rejected records plus reasons.
- State advances only after the full run succeeds.

Try / inspect

```
source -> landing/run_id/raw -> staging
                        -> quarantine
                        -> state after SUCCESS
```

Common failure

Writing directly into staging/serving and losing the original source evidence.

Your turn - independent proof

Draw the architecture for a new vendor feed and name exactly when state can advance.

Move on when

You can trace one source record through raw, staged/rejected and checkpoint outcomes.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L02 - Defensive CSV ingestion

Learning objective

Parse CSVs using contracts and quarantine bad rows instead of assuming every file is clean.

Why this matters in Data Engineering

CSV is common but weakly typed; headers, quoting, missing values and duplicate keys must be validated.

Understand

- Use a CSV parser, not naive string splitting.
- Check required headers before record processing.
- Keep source row number/file/run ID for traceability.
- Separate record defects from whole-file contract failures.

Try / inspect

```
python -m src.ingest_csv_reference source/files/orders_2026-08-01.csv  
python -m src.ingest_csv_reference source/files/orders_bad.csv
```

Common failure

Dropping bad rows silently and reporting a clean batch count.

Your turn - independent proof

Build ingest_csv_TODO.py: good file -> staged; bad file -> quarantine with reasons.

Move on when

Every input row has an explainable staged/quarantined outcome.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L03 - JSON and nested-record ingestion

Learning objective

Flatten nested records only after declaring the output grain.

Why this matters in Data Engineering

One JSON object can contain arrays that produce several output rows; grain must be explicit to prevent accidental duplication.

Understand

- Objects map to dictionaries; arrays map to lists.
- State the output grain before exploding nested arrays.
- Preserve parent identifiers and event/source timestamps.
- Do not flatten away information needed for lineage.

Try / inspect

```
order {customer:{...}, items:[{...},{...}]}  
# staging grain = one row per order item
```

Common failure

Flattening nested arrays without preserving parent keys, creating ambiguous rows.

Your turn - independent proof

Flatten orders_nested.json and prove J3001 becomes exactly two item-grain rows.

Move on when

You can state source object grain and staging row grain and explain the row-count change.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L04 - Parquet ingestion and metadata inspection

Learning objective

Use columnar metadata/types deliberately rather than treating Parquet as compressed CSV.

Why this matters in Data Engineering

Parquet stores typed columnar data and useful metadata that can avoid full-file scans.

Understand

- Inspect schema and row count before reading everything when possible.
- Column types are persisted more strongly than CSV text.
- Partition/file layout affects later engines even if the local file is tiny.
- Do not assume writer row-group choices are universal.

Try / inspect

```
python -m src.parquet_lab
# inspect schema, row count, selected columns
```

Common failure

Converting CSV to Parquet but preserving wrong inferred types and calling it solved.

Your turn - independent proof

Convert the packaged 8-row file, inspect metadata/types and read selected columns only.

Move on when

You can state what metadata proves without loading the whole dataset.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L05 - REST API contracts, status codes and timeouts

Learning objective

Call HTTP APIs with explicit success/error boundaries and bounded waits.

Why this matters in Data Engineering

A network call can fail before data quality begins; 200 status does not prove payload correctness.

Understand

- Set connect/read timeouts.
- Interpret 2xx, 4xx and 5xx classes intentionally.
- Call `raise_for_status` or equivalent before trusting payload.
- Validate payload schema after transport succeeds.

Try / inspect

```
requests.get(url, headers=headers, timeout=(3,15))
r.raise_for_status()
data=r.json()
```

Common failure

No timeout, or treating any JSON body as success even when status is an error.

Your turn - independent proof

Use the offline API to prove 200/401/404 behavior and reject malformed success payloads.

Move on when

Transport success and data-contract success are checked separately.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L06 - API authentication, pagination and rate limits

Learning objective

Keep credentials out of code and retrieve all pages without assuming page count.

Why this matters in Data Engineering

APIs commonly paginate and throttle; missing pages is silent data loss.

Understand

- Read tokens from environment/secret store; never log them.
- Follow next token/page metadata rather than hard-coding total pages.
- Respect Retry-After on 429 where provided.
- Record page identifiers/counts for completeness evidence.

Try / inspect

```
headers={"Authorization":"Bearer "+os.environ["TRAINING_API_KEY"]}
while next_page: fetch(next_page)
```

Common failure

Embedding a real token in source or stopping after page 1 because the response looks valid.

Your turn - independent proof

Fetch all 10 v1 records at page_size=3 and handle a simulated page-2 429 without exposing the token.

Move on when

Page counts and final record count prove pagination completeness.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L07 - Retries, backoff, jitter and transient failures

Learning objective

Retry only failures likely to recover and keep attempts bounded.

Why this matters in Data Engineering

Retries can improve reliability or amplify outages/duplicates if applied blindly.

Understand

- Retry connection failures, selected 5xx and 429 by policy.
- Do not repeatedly retry deterministic 400/401 errors.
- Exponential backoff spaces attempts; jitter reduces synchronized retry storms.
- Idempotency matters before retrying writes.

Try / inspect

```
for attempt in range(1,5):  
    wait = 0.5 * (2 ** (attempt-1))  
    # add bounded jitter / honor Retry-After
```

Common failure

Infinite retry loops or retrying authentication failures that require configuration repair.

Your turn - independent proof

Trigger one-time 500 then success; prove 401 is not blindly retried.

Move on when

Attempt log shows bounded retries and final outcome.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L08 - Database extraction and safe query boundaries

Learning objective

Extract read-only bounded data with deterministic ordering and explicit columns.

Why this matters in Data Engineering

Source databases are operational systems; ingestion should minimize load and never mutate them accidentally.

Understand

- Use read-only credentials/transactions when possible.
- Avoid unbounded SELECT * for repeated extraction.
- Use deterministic ordering and development LIMIT.
- High-water predicates need a tie-breaker for equal timestamps.

Try / inspect

```
WHERE updated_at > :ts
      OR (updated_at=:ts AND order_id>:id)
ORDER BY updated_at, order_id
LIMIT :n
```

Common failure

Timestamp-only incremental query that can skip rows sharing the same update time.

Your turn - independent proof

Extract rows after watermark (08:15,D5002) and prove the returned IDs/order.

Move on when

The query is read-only, bounded and deterministic under tied timestamps.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L09 - Historical batch landing and raw metadata

Learning objective

Land historical files immutably with run/file metadata before transformation.

Why this matters in Data Engineering

Backfills and audits require knowing exactly which bytes were processed in each run.

Understand

- Use unique run directories or immutable object paths.
- Capture filename, bytes, checksum and retrieval/run time.
- Do not overwrite a prior raw object with a later file of the same name.
- Keep source identity separate from processing status.

Try / inspect

```
landing/run_20260827/orders_2026-08-01.csv  
_manifest.json -> sha256, bytes, source_name
```

Common failure

Overwriting old raw batches and later being unable to reproduce a historical run.

Your turn - independent proof

Land both historical files unchanged and generate a checksum manifest.

Move on when

A raw file can be matched unambiguously to a run and checksum.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L10 - Incremental loads and high-water marks

Learning objective

Advance a composite checkpoint only after successful complete processing.

Why this matters in Data Engineering

A checkpoint is a promise that everything before a position is handled; advancing early can permanently skip data.

Understand

- Use (updated_at,business_key) when timestamps can tie.
- Fetch/land/normalize/write before committing state.
- Keep state update atomic with success where possible.
- Record previous/new checkpoints in run metadata.

Try / inspect

```
checkpoint=("2026-08-05T09:00:00Z", "A4008")  
# state written only after all pages + staging succeed
```

Common failure

Updating state after page 1 and then failing on page 2, causing missed records next run.

Your turn - independent proof

Process the normal API feed and verify final checkpoint includes timestamp + order_id tie-breaker.

Move on when

You can prove why every row $\leq$ checkpoint has been fully handled.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L11 - CDC concepts and change-event semantics

Learning objective

Interpret insert/update/delete change events without confusing event order with table snapshots.

Why this matters in Data Engineering

Change Data Capture is a source-interface pattern; reliable consumption depends on identity, ordering and replay semantics.

Understand

- CDC emits changes rather than full snapshots.
- Events need stable source identity/order/offset semantics.
- Updates may contain before/after images or partial fields depending on source.
- Deletes need explicit tombstone/delete handling; ignoring them creates stale targets.

Try / inspect

```
event={op:"u", key:"07", before:{...}, after:{...}, source_pos:"..."}
```

Common failure

Applying change events twice or out of order without an idempotent key/position rule.

Your turn - independent proof

Given a small CDC event file, derive final current state and record which source positions were applied.

Move on when

You can explain insert/update/delete events, replay and the ordering key.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L12 - Schema drift, compatibility, time zones and quarantine

Learning objective

Detect incompatible source changes and keep temporal meaning explicit.

Why this matters in Data Engineering

Schema drift can silently map wrong columns/types; local-time ambiguity can reorder events.

Understand

- Compare actual vs expected fields/types.
- Classify compatible additions separately from breaking rename/type changes.
- Normalize timestamps to an agreed timezone while preserving raw value.
- Quarantine/stop incompatible data rather than guessing.

Try / inspect

```
expected={order_id,updated_at,customer_id,unit_price}  
# v2 renames unit_price -> price => explicit drift decision
```

Common failure

Auto-accepting renamed/retargeted fields and producing plausible values under the wrong contract.

Your turn - independent proof

Feed the v2 drift payload and prove strict v1 normalization rejects/quarantines it with a reason.

Move on when

No incompatible schema change reaches staging silently.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L13 - Checkpoint/state management and replay safety

Learning objective

Make state explicit, durable and separate from raw evidence; support replay without accidental advancement.

Why this matters in Data Engineering

Reprocessing is normal for repair/backfill. State must not make a replay irreproducible or duplicate outputs.

Understand

- Store checkpoint plus source/run metadata.
- Normal run may advance state; replay/backfill defaults to no state advance unless explicitly promoted.
- Deduplicate at stable business/event identity.
- Make state writes atomic or recoverable.

Try / inspect

```
normal: land -> stage -> commit state  
replay: land -> stage -> validate -> NO state commit by default
```

Common failure

Replaying a historical run and unintentionally moving the production checkpoint backward/forward.

Your turn - independent proof

Replay an already processed batch into a fresh run ID and prove business keys do not duplicate and normal checkpoint is unchanged.

Move on when

Replay/backfill behavior is explicit and state transition evidence is saved.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L14 - Idempotent landing and content identity

Learning objective

Use content/file identity to distinguish safe reruns from genuinely new source material.

Why this matters in Data Engineering

Same filename can contain different bytes; different filenames can contain the same batch.

Understand

- Compute checksum/size at landing.
- Choose whether duplicate content should skip, reupload, or reprocess based on policy.
- Never use filename alone as proof of identity.
- Keep content identity separate from logical business-key deduplication.

Try / inspect

```
sha=sha256(file_bytes)
if sha in processed and not replay: skip
```

Common failure

Skipping a changed file because its filename was seen before.

Your turn - independent proof

Create two files with same name/different content across run folders and two names/same content; apply the declared identity policy.

Move on when

You can explain source-object identity vs business-record identity.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L15 - Late-arriving data, event time and processing time

Learning objective

Handle data that arrives after its business/event time without corrupting windows/checkpoints.

Why this matters in Data Engineering

Ingestion time and event time answer different questions. Late data is common in batch/API systems.

Understand

- Preserve event timestamp and ingestion timestamp separately.
- Watermarks based only on processing time may miss corrected/late events.
- Define allowed lateness/backfill policy for downstream systems.
- Do not silently replace event time with current time.

Try / inspect

```
event_time=record["updated_at"]
ingested_at=run_started_at
# keep both
```

Common failure

Sorting by ingestion time when the business logic requires event chronology.

Your turn - independent proof

Inject one older event in a later batch and document how it is landed, staged and made eligible for downstream repair.

Move on when

You can state which timestamp drives business ordering, checkpointing and operational monitoring.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L16 - Ingestion observability: counts, latency and run metadata

Learning objective

Emit enough evidence to know what arrived, what failed and how long each boundary took.

Why this matters in Data Engineering

A green process with missing pages/rows is still wrong. Operational evidence must cover completeness and failure state.

Understand

- Record source objects/pages, bytes, raw rows, staged rows, quarantined rows and checkpoint.
- Track start/end/duration and status.
- Never log secrets or full sensitive payloads unnecessarily.
- Make run IDs connect logs, landing and state.

Try / inspect

```
run={id,status,pages,raw_rows,staged_rows,quarantine_rows,checkpoint,duration_s}
```

Common failure

Logging only "completed successfully" with no completeness counters.

Your turn - independent proof

Add a structured run manifest to one ingestion path and use it to diagnose an intentionally failed page.

Move on when

A reviewer can reconstruct the run outcome from metadata without reading the source code.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.

M08-L17 - Ingestion integration, reconciliation and backfill proof

Learning objective

Operate the whole ingestion path and prove source-to-staging completeness under normal, failure and replay conditions.

Why this matters in Data Engineering

The module ends with behavior across interfaces, not isolated parser exercises.

Understand

- Reconcile file/API/database source identities and row counts.
- Every rejected record has a reason and is preserved.
- Checkpoint advances only after success.
- Backfill/replay produces controlled equivalent business state without duplicates.

Try / inspect

```
source rows/pages -> landed -> staged + quarantined
state only after SUCCESS
replay -> stable business keys
```

Common failure

Passing each small lab but failing end-to-end completeness or recovery.

Your turn - independent proof

Run a mixed file+API scenario, inject one transient failure and one bad record, recover, replay and produce a reconciliation report.

Move on when

The end-to-end evidence proves completeness, quarantine, state and rerun behavior.

Revision cadence: D+1 fresh retrieval; D+4 mixed task; D+10 repair scenario; D+30 interview retrieval.